

Ex4

4.1 Tool Description

Valgrind is a fully automatic dynamic analysis framework that instruments programs at runtime to monitor their behaviour. We used its Memcheck tool applied to the hashmap.c code. Compared to the tools in Exercises 1–3, Valgrind takes a different approach. Infer and Clang Static Analyzer, used in Exercise 1, never run the program. The fuzzer in Exercise 2 runs the program but focuses on generating inputs that trigger crashes. Symbolic execution in Exercise 3 explores possible paths mathematically, without committing to concrete values. Valgrind, by contrast, simply runs the program and watches what happens at the memory level, instruction by instruction. This makes it very good at catching concrete memory bugs: invalid reads and writes, heap buffer overflows, use-after-free, double frees, memory leaks, and uses of uninitialised memory. The main limitation is coverage: Valgrind can only catch bugs that the test inputs actually trigger. This is why it works best alongside static and symbolic tools, rather than as a standalone solution.

4.2 Tool Application

For the first execution, the original input in `main` was used, where the key is "42". This execution detected a memory leak related to the return value of `hashmap_get`.

```
[base] ~/workspace/learned-lex-3-14611: hashmap/03/03_03_03/HashMap$ valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./hashmap
==38646== Memcheck, a memory error detector
==38646== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==38646== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==38646== Command: ./hashmap
==38646==
==38646== HEAP SUMMARY:
==38646==   in use at exit: 4 bytes in 1 blocks
==38646== total heap usage: 6 allocs, 5 frees, 184 bytes allocated
==38646==
==38646== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==38646==   at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==38646==   by 0x109797: hashmap_get (hashmap.c:155)
==38646==   by 0x109839: main (hashmap.c:169)
==38646==
==38646== LEAK SUMMARY:
==38646==   definitely lost: 4 bytes in 1 blocks
==38646==   indirectly lost: 0 bytes in 0 blocks
==38646==   possibly lost: 0 bytes in 0 blocks
==38646==   still reachable: 0 bytes in 0 blocks
==38646==   suppressed: 0 bytes in 0 blocks
==38646==
==38646== For lists of detected and suppressed errors, rerun with: -s
==38646== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Figure 1: Valgrind ex1 1st execution

Figure 2 shows that, after replacing the original key "42" with a longer key "this_is_a_long_key", Valgrind reported invalid write and invalid read

errors during the execution of the program. The different report occurs because Valgrind is a dynamic-analysis tool and therefore only detects bugs that are exercised by the executed input. With the original short key, the incorrect allocation in `hashmap_set` did not visibly overflow the allocated memory block. However, when a longer key was used, the call to `strcpy` wrote beyond the allocated space, causing invalid writes, and later `strcmp` attempted to read beyond the same invalid memory region, causing invalid reads.

```

==43025== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==43025== Command: ./hashmap
==43025==
==43025== Invalid write of size 1
==43025== at 0x484EE7C: strcpy (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x1095F5: hashmap_set (hashmap.c:125)
==43025== by 0x109826: main (hashmap.c:166)
==43025== Address 0x4a9a1b8 is 0 bytes after a block of size 8 alloc'd
==43025== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x1095D5: hashmap_set (hashmap.c:124)
==43025== by 0x109826: main (hashmap.c:166)
==43025==
==43025== Invalid write of size 1
==43025== at 0x484EE8E: strcpy (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x1095F5: hashmap_set (hashmap.c:125)
==43025== by 0x109826: main (hashmap.c:166)
==43025== Address 0x4a9a1c2 is 10 bytes after a block of size 8 alloc'd
==43025== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x1095D5: hashmap_set (hashmap.c:124)
==43025== by 0x109826: main (hashmap.c:166)
==43025==
==43025== Invalid read of size 1
==43025== at 0x484FB88: strcmp (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x109783: hashmap_get (hashmap.c:154)
==43025== by 0x109839: main (hashmap.c:168)
==43025== Address 0x4a9a1b8 is 0 bytes after a block of size 8 alloc'd
==43025== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x1095D5: hashmap_set (hashmap.c:124)
==43025== by 0x109826: main (hashmap.c:166)
==43025==
==43025== HEAP SUMMARY:
==43025==   in use at exit: 4 bytes in 1 blocks
==43025== total heap usage: 6 allocs, 5 frees, 184 bytes allocated
==43025==
==43025== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==43025== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==43025== by 0x109797: hashmap_get (hashmap.c:155)
==43025== by 0x109839: main (hashmap.c:168)
==43025==
==43025== LEAK SUMMARY:
==43025==   definitely lost: 4 bytes in 1 blocks
==43025==   indirectly lost: 0 bytes in 0 blocks
==43025==   possibly lost: 0 bytes in 0 blocks
==43025==   still reachable: 0 bytes in 0 blocks
==43025==   suppressed: 0 bytes in 0 blocks
==43025==
==43025== For lists of detected and suppressed errors, rerun with: -s
==43025== ERROR SUMMARY: 23 errors from 4 contexts (suppressed: 0 from 0)

```

Figure 2: Valgrind ex1 2nd execution

Overall, Valgrind did not reveal additional bugs beyond those already identified in Exercise 1 by Infer and clang-analyzer; however, it dynamically confirmed the incorrect key allocation and the memory leak in `main`, showing how these bugs manifest during concrete executions of the program.

```

==486760== Address 0x4a9a1b8 is 0 bytes after a block of size 8 alloc'd
==486760== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==486760== by 0x1095D5: hashmap_set (hashmap.c:124)
==486760== by 0x109826: main (hashmap.c:166)
==486760==

```

Figure 3: Allocation trace reported by Valgrind

Figure 3 shows Valgrind tracing the invalid memory access back to the allocation performed in `hashmap_set`. The report states that the invalid address is im-

mediately after a block of only 8 bytes allocated at line 124. This confirms that the key buffer was too small, because the program used `malloc(sizeof(key))`, where `key` is a pointer, instead of allocating enough space for the full string.